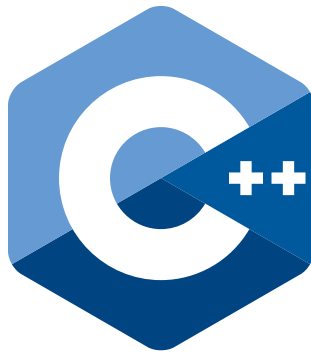




Programming with C++

Skylar Coelho

March 3, 2026

Most content taken from www.learncpp.com

Contents

1 C++ Basics	1
1.1 Comments	1
1.2 Variable Declaration and Assignment	1
1.3 Common Types	2
2 Functions	2
2.1 Function Declaration	2
2.2 Function Parameters	3

1 C++ Basics

1.1 Comments

Single line comments are done with `"/"`

```
1      std::cout << "Hello world!"; // Everything from here to the end of the line is ignored
```

Multi-line comments are done with `"/*...*/"`

```
1      /* This is a multi-line comment.
2         * the matching asterisks to the left
3         * can make this easier to read
4         */
```

1.2 Variable Declaration and Assignment

Variables are declared using their type and identifier

```
1      int main()
2      {
3          int x;    // define an integer variable named x (preferred)
4          int y, z; // define two integer variables, named y and z
5
6          return 0;
7      }
```

Variables do not need a value when they are declared, but *can* be given one if it makes sense

Table 1: Common data types in C++

Group	Type names*	Notes on size / precision
Character types	char	Exactly one byte in size. At least 8 bits.
	char16_t	Not smaller than char. At least 16 bits.
	char32_t	Not smaller than char16_t. At least 32 bits.
	wchar_t	Can represent the largest supported character set.
Integer types (signed)	signed char	Same size as char. At least 8 bits.
	signed short int	Not smaller than char. At least 16 bits.
	signed int	Not smaller than short. At least 16 bits.
	signed long int	Not smaller than int. At least 32 bits.
Integer types (unsigned)	signed long long int	Not smaller than long. At least 64 bits.
	unsigned char	(same size as their signed counterparts)
	unsigned short int	
	unsigned int	
	unsigned long int	
Floating-point types	unsigned long long int	
	float	
	double	Precision not less than float
	long double	Precision not less than double
Boolean type	bool	
Void type	void	no storage
Null pointer	decltype(nullptr)	

```

1      int a;           // default-initialization (no initializer)
2
3      // Traditional initialization forms:
4      int b = 5;        // copy-initialization (initial value after equals sign)
5      int c ( 6 );      // direct-initialization (initial value in parenthesis)
6
7      // Modern initialization forms (preferred):
8      int d { 7 };      // direct-list-initialization (initial value in braces)
9      int e {};         // value-initialization (empty braces)

```

1.3 Common Types

2 Functions

2.1 Function Declaration

Functions are declared with a return type and an identifier, along with the arguments they take

```

1      returnType functionName() /* This is the function header
2      (tells the compiler about the existence of the function)*/
3      {
4          // This is the function body (tells the compiler what the function does)
5      }

```

Functions can call other functions, but a function can not be defined within another one. Functions can either return

a value, or not (void). Main must always return an integer (int) which is known as a status code that can be used to indicate whether or not the code performed as it should have.

2.2 Function Parameters

Function parameters are defined when the function is declared

```
1      // This function takes no parameters
2      // It does not rely on the caller for anything
3      void doPrint()
4      {
5          std::cout << "In doPrint()\n";
6      }
7
8      // This function takes one integer parameter named x
9      // The caller will supply the value of x
10     void printValue(int x)
11     {
12         std::cout << x << '\n';
13     }
14
15     // This function has two integer parameters, one named x, and one named y
16     // The caller will supply the value of both x and y
17     int add(int x, int y)
18     {
19         return x + y;
20     }
```

These pre-defined values are known as *parameters*. Values passed to a function are known as *arguments*.